UNIVERSIDAD DISTRITAL FRANCISCO JOSÉ DE CALDAS
Facultad de Ingeniería – Proyecto Curricular de Ingeniería de Sistemas
Grupo de Investigación Multimedia Interactiva y Animación Digital

INFORME TÉCNICO DE AVANCE Y DESARROLLO

HITO 2: Documentación de Diseño, Arquitectura y Requerimientos AR

Gamificación y Seguimiento de Rutas en Entorno AR-Sandbox

Proyecto:	AR-Sandbox: Seguimiento de Rutas	**Periodo:**	2026-III
Hito / Avance:	Hito 2 de 7	**Fecha de corte:**	22 de septiembre, 2026
Presentado por:	Joshoa Alarcón Sánchez (20221020013)	**Repositorio Base:**	`thomwolf/magic-sand`

1. Objetivos del Hito de Entrega

El presente documento expone las decisiones de diseño de software, la especificación formal de requerimientos y el Game Design Document (GDD v1.0) para la incorporación de mecánicas lúdicas sobre la plataforma física AR-Sandbox, partiendo de la base de código libre `Magic-Sand`.

Los objetivos específicos correspondientes a este hito son:

- Definir los requisitos funcionales y no funcionales orientados a la interacción física mediante manipulación tangible de la arena.
- Justificar la arquitectura de software en C++ nativo (*openFrameworks*) frente a la integración de motores externos.
- Diseñar el flujo lúdico estructurado en dos fases consecutivas: Escultura Topográfica y Juego Activo.
- Elaborar el Game Design Document (GDD) detallando mecánicas, niveles temáticos, penalizaciones y el sistema de puntuación.
- Formular la estructura modular de clases en C++ requerida para la implementación operativa del Hito 3.

2. Arquitectura del Sistema y Justificación Técnica

2.1. Decisión de Arquitectura: Motor Externo (Godot) vs. C++ Nativo

Se evaluó desacoplar la gamificación hacia un motor externo como Godot mediante memoria compartida (Spout). No obstante, dado que la interacción no demanda física de cuerpos rígidos 3D sino la **evaluación matricial de alturas en tiempo real**, se determinó mantener el desarrollo de forma monolítica y modular en C++ sobre *openFrameworks*:

- **Latencia mínima:** El procesamiento del búfer del Kinect y la lógica lúdica operan en el mismo hilo, evitando retardos de sincronización entre procesos (< 16 ms por cuadro).
- **Reutilización de calibración:** Se aprovecha la matriz de homografía $\mathbf{H}_{3\times3}$ y el viewport ya configurados en la caja de arena.
- **Eficiencia:** Elimina la necesidad de serializar y transmitir matrices bidimensionales densas entre procesos heterogéneos.

2.2. Línea Base del Entorno AR-Sandbox

La estación de trabajo cuenta con una serie de subsistemas previamente estabilizados:

- Calibración proyectiva y homografía cámara-proyector completamente operativas.
- Captura de profundidad cruda mediante Kinect v1/v2 funcional.
- Shader hipsométrico operativo con 4 estratos topográficos de color: **Azul** (depresión/agua), **Verde** (valle transitable), **Amarillo** (colina/obstáculo) y **Rojo** (cima/muro).
- **Restricción de control:** Se prescinde del rastreo de manos o dedos. Toda interacción se rige por la modificación física del relieve de la arena.

Directriz de Diseño de Interacción Tangible

La arena actúa simultáneamente como controlador de entrada y pantalla física de salida. El avance de rutas y la activación de eventos dependen únicamente de las transiciones de cota generadas al apilar o retirar arena en las zonas proyectadas.

2.3. Arquitectura Modular del Software y Hardware

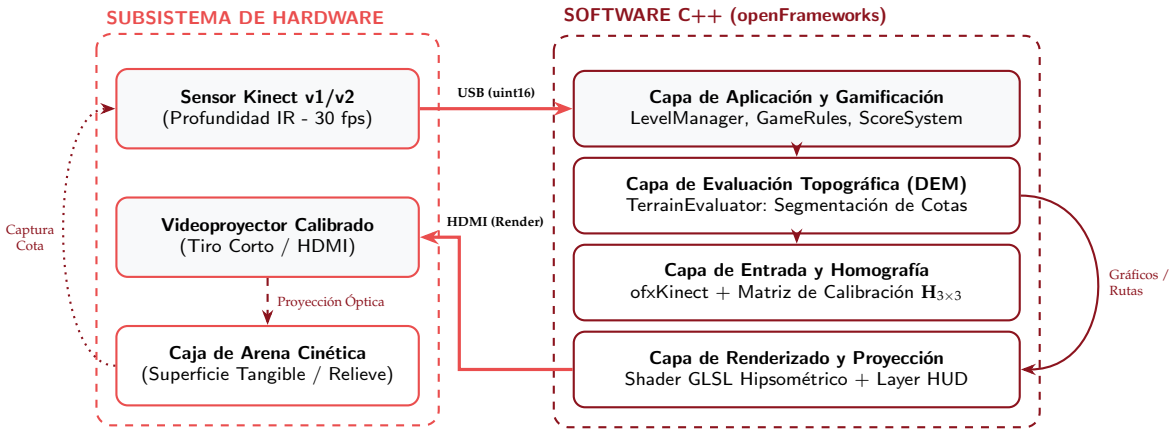


Figura 1: Arquitectura general del sistema: acoplamiento entre el subsistema de hardware de la mesa física y las capas desacopladas en C++ sobre *openFrameworks*.

2.4. Tubería de Procesamiento por Fotograma (*Pipeline* de Datos)

El ciclo de ejecución en cada fotograma sigue una secuencia determinista para sostener tasas superiores a 30 fps sin bloqueos de memoria:

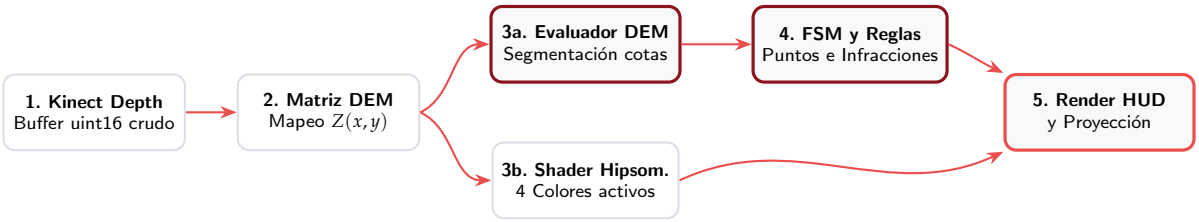


Figura 2: Tubería determinista de procesamiento sensorial por fotograma: bifurcación entre el sombreado hipsométrico visual y la máquina de evaluación de reglas.

3. Requerimientos del Sistema y Reglas de Terreno

3.1. Requerimientos Funcionales (RF)

- **RF-01 (Proyección de Plantilla):** El sistema debe proyectar sobre la arena las guías visuales del nivel (contornos de muros o zanjas) durante la fase inicial de modelado.
- **RF-02 (Conformidad de Terreno):** El sistema debe contrastar la matriz de elevación capturada contra la plantilla objetivo y cuantificar el porcentaje de similitud.
- **RF-03 (Transición Automática):** Debe iniciar automáticamente la fase activa cuando el terreno coincida en un $\geq 80\,\%$ con la plantilla durante 2 segundos sostenidos.
- **RF-04 (Dinámica de Flujo por Relieve):** El elemento virtual avanza únicamente por zonas Verdes, se detiene ante muros Amarillos/Rojos y cae si ingresa a zonas Azules no autorizadas.
- **RF-05 (Detección Pasiva sin Manos):** La evaluación de victoria, derrota o desvío se calcula estrictamente con base en las cotas de la arena.

- **RF-06 (Control de Infracciones):** El sistema debe identificar derrumbes de muros, desbordamientos o cortes de camino, penalizando el puntaje en tiempo real.
- **RF-07 (HUD Perimetral):** El sistema proyecta información marginal: tiempo transcurrido, puntaje actual y alertas contextuales.

3.2. Requerimientos No Funcionales (RNF)

- **RNF-01 (Rendimiento):** El ciclo completo de lectura de terreno, lógica de reglas y renderizado proyectado debe sostenerse a ≥ 30 fps.
- **RNF-02 (Latencia de Detección):** La respuesta ante una modificación topográfica en la arena debe reflejarse en la lógica del juego en menos de 100 ms.
- **RNF-03 (Modularidad):** La lógica del evaluador de terreno debe permitir pruebas aisladas con matrices sintéticas sin requerir el sensor Kinect conectado.

3.3. Significado Lúdico de los Estratos Topográficos

El sistema traduce los 4 rangos de elevación preconfigurados en mecánicas funcionales:

Color	Rango de Cota Física	Mecánica / Comportamiento de Juego
Azul	Profundidad máxima / Suelo	**Agua / Abismo:** Infranqueable para entidades terrestres. Sirve de canal para navíos o riesgo si inunda valles protegidos.
Verde	Depresión media / Valle	**Camino Transitable:** Único estrato que permite el paso libre de avatares, flujos guiados y vehículos de exploración.
Amarillo	Altura intermedia / Terraplén	**Contención Secundaria:** Ralentiza el paso o actúa como aviso previo a muros altos.
Rojo	Elevación máxima / Cima	**Muralla / Obstáculo:** Bloqueo absoluto. Delimita los muros del laberinto o diques de contención.

4. Game Design Document (GDD v1.0) – TerraMaze AR

4.1. Premisa y Bucle Principal de Juego (*Core Loop*)

En *TerraMaze AR*, el jugador actúa como un ingeniero de terreno que debe moldear físicamente la arena para resolver acertijos espaciales y guiar elementos virtuales desde un origen hasta su destino.

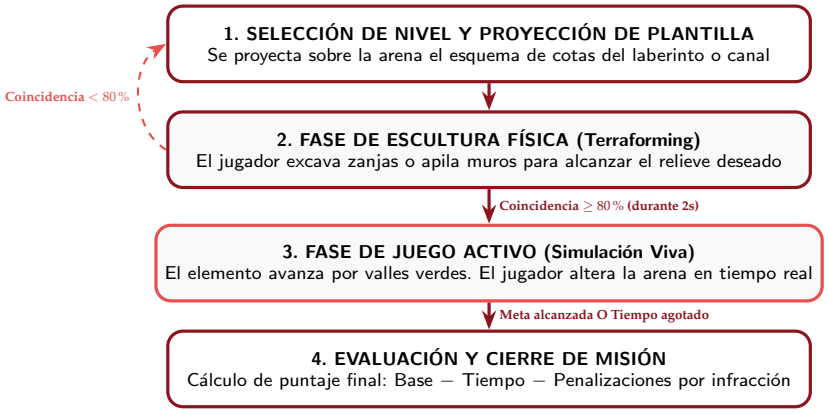


Figura 3: Diagrama de la Máquina de Estados Finitos (FSM) que gobierna el bucle de juego interactivo en la AR-Sandbox.

4.2. Diseño Detallado de Casos y Misiones

4.2.1. Caso 1: El Canal de Navegación (Introductorio / Cavado de Zanjas)

- **Fase de Escultura:** Se proyectan dos franjas paralelas en zigzag. El jugador debe retirar arena hasta que el fondo del canal se torne **Azul (Agua)**, manteniendo las riberas en **Verde**.
- **Fase de Juego:** Un barco virtual zarpa desde el muelle de origen avanzando por el canal.

- **Mecánica Dinámica:** Se proyectan obstrucciones en el agua; el jugador debe retirar arena rápidamente para despejar la ruta antes de que el barco encalle.
- **Infracciones:** Si la arena colapsa hacia el canal y el agua desaparece bajo el barco (encallamiento: -100 pts por ocurrencia).

4.2.2. Caso 2: El Laberinto de las Murallas (Intermedio / Apilado de Muros)

- **Fase de Escultura:** Se proyecta el esquema de un laberinto. El jugador levanta crestas de arena sobre las líneas proyectadas hasta alcanzar el nivel **Rojo (Cima)**, dejando los pasillos despejados en nivel **Verde (Valle)**.
- **Fase de Juego:** Exploradores virtuales avanzan automáticamente por los corredores verdes.
- **Mecánica Dinámica:** Se simulan derrumbes en secciones del laberinto; el jugador debe allanar muros o abrir nuevas brechas verdes para reconducir la marcha hacia la salida.
- **Infracciones:** Si un muro pierde altura por debajo del nivel Amarillo, los exploradores se desvían de la ruta autorizada (fuga: -50 pts/segundo).

4.2.3. Caso 3: La Contención del Valle (Avanzado / Gestión de Diques)

- **Fase de Escultura:** Se proyecta un valle central (Verde) que aloja un poblado virtual, rodeado de áreas perimetrales de riesgo donde el jugador debe construir diques en **Rojo**.
- **Fase de Juego:** Se activa una fuente de inundación en un extremo del terreno.
- **Mecánica Dinámica:** El jugador refuerza físicamente los diques de arena y abre canales hacia fosos laterales (Azul) para desviar el caudal lejos del poblado.
- **Condición de Victoria:** Resistir la inundación durante 60 segundos manteniendo el poblado a salvo.
- **Infracciones:** Cada segundo que el agua virtual cubra celdas del valle protegido resta -30 pts.

4.3. Sistema de Puntuación y Deducciones

El cálculo del desempeño final no emplea formulaciones matemáticas complejas sino una estructura lógica transparente:

$$\text{Puntaje Final} = \text{máx}\left(0,\ S_{\text{base}} - (\Delta t \times C_t) - \sum \text{Deducciones por Infracción}\right) \tag{1}$$

- **Puntaje Base (S_{base}):** 1000 puntos otorgados al superar el objetivo del nivel.
- **Penalización Temporal (C_t):** Deducción de 5 puntos por segundo consumido. En tiempo extra, la tasa asciende a 15 puntos/segundo.
- **Deducciones por Infracción:** Encallamiento o rotura de dique (-100 pts); desvío de ruta (-50 pts); inundación de área protegida (-30 pts/segundo).
- **Escala de Calificación:** 3 Estrellas (≥ 750 pts), 2 Estrellas ($500 - 749$ pts), 1 Estrella ($250 - 499$ pts), No Superado (< 250 pts).

4.4. Diseño del HUD Proyectado sobre la Arena

La interfaz gráfica (HUD) se proyecta sobre las zonas marginales de la caja de arena para no entorpecer el área activa de modelado físico:

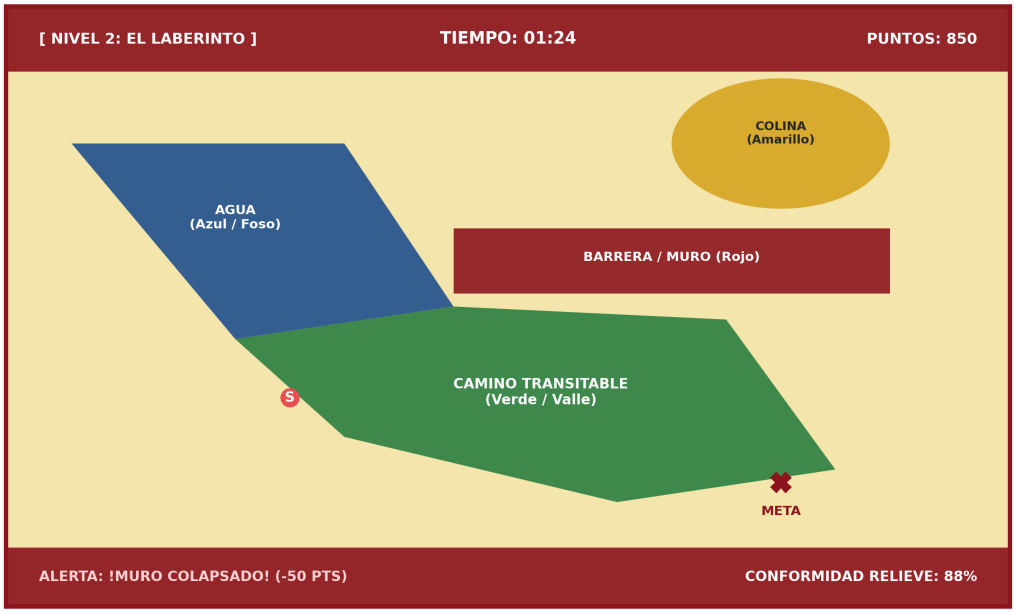


Figura 4: Distribución espacial de la proyección: bandas de información marginal del HUD y zona central activa de interacción tangible sobre la arena.

5. Estructura Computacional de Clases en C++

Para la materialización técnica en el Hito 3, se estructuran módulos acoplados al ciclo de vida estándar de *openFrameworks* (`setup`, `update`, `draw`), organizados en la cabecera directriz del gestor de niveles:

```
#pragma once
#include "ofMain.h"

// Estados finitos del ciclo de juego en la AR-Sandbox
enum GameState {
    STATE_SCULPTING,   // Proyeccion de plantilla y moldeado manual de arena
    STATE_PLAYING,     // Fase activa: navegacion sobre relieve y conteo de reglas
    STATE_LEVEL_CLEAR, // Nivel completado, despliegue de calificacion y estrellas
    STATE_GAME_OVER    // Puntos agotados o tiempo excedido
};

// Bandas discretas de altura correspondientes al shader hipsometrico
enum TopoZone {
    ZONE_WATER = 0,   // Azul: cota minima (canal navegable o foso)
    ZONE_VALLEY = 1,  // Verde: camino transitable para entidades
    ZONE_HILL = 2,    // Amarillo: pendiente / contencion secundaria
    ZONE_PEAK = 3     // Rojo: muro o barrera infranqueable
};

// Configuracion de mision parametrizable
struct LevelConfig {
    int levelId;
    string levelName;
    float timeLimitSeconds;
    int baseScore;
    vector<int> targetTopography; // Matriz discretizada de zonas objetivo
};

class LevelManager {
public:
    LevelManager();
    void setup(int resolutionX, int resolutionY);
    void update(const ofFloatPixels& currentDepthMap);
    void drawHUD();
    void drawLevelTemplate();

    GameState getCurrentState() const { return currentState; }
    float getConformityPercentage() const { return conformityPercent; }
    int getCurrentScore() const { return currentScore; }
```

```
private:
    GameState currentState;
    LevelConfig currentLevel;
    float conformityPercent;
    int currentScore;
    float stateTimer;

    void evaluateTerrain(const ofFloatPixels& currentDepthMap);
    void processGameRules();
};
```

Listing 1: Estructura de clases y tipos de la máquina de estados lúdica (LevelManager.h).

6. Criterios de Aceptación y Estado del Hito

6.1. Matriz de Verificación del Hito 2

Componente / Requisito	Criterio de Aceptación	Meta	Estado
SRS y Requerimientos	Requisitos funcionales adaptados a interacción sin manos	100 %	**Conforme**
Arquitectura de Software	Justificación técnica C++ vs. Godot y diseño modular	Documentado	**Conforme**
GDD v1.0	Definición de mecánicas, 3 casos lúdicos y scoring	3 Niveles	**Conforme**
Diseño de HUD Proyectado	Distribución no invasiva de interfaz sobre arena	Especificado	**Conforme**
Estructura de Clases C++	Declaración de módulos para implementación en Hito 3	Código base	**Conforme**

6.2. Compromisos para el Siguiente Hito Quincenal (Hito 3)

Conforme a la planificación operativa quincenal, los compromisos técnicos inmediatos corresponden a:

1. **Compromiso 1:** Implementar la clase `TerrainEvaluator` en C++ para clasificar las cotas del Kinect en las 4 bandas discretas (Azul, Verde, Amarillo, Rojo).
2. **Compromiso 2:** Programar el comparador matricial entre el relieve esculpido físicamente y la plantilla proyectada del Nivel 1.
3. **Compromiso 3:** Proyectar la capa visual del HUD preliminar mostrando el porcentaje de similitud del terreno en tiempo real.

7. Recursos Computacionales y Reproducibilidad

- **Entorno de Desarrollo:** C++11/C++14, framework *openFrameworks* (v0.10.x / v0.11.x).
- **Addons y Librerías:** `ofxKinect`, `ofxOpenCv`, `ofxGui`.
- **Hardware Integrado:** Mesa de arena cinética, Sensor Microsoft Kinect (v1 Xbox 360 / v2), Videoproyector de tiro corto calibrado mediante homografía.
- **Estructura en Repositorio:** `src/modules/LevelManager.h`, `src/modules/TerrainEvaluator.h`.

8. Referencias

[1] Reed, K. E., et al. (2014). *Shaping the future of augmented reality in science education: The Augmented Reality Sandbox*. UC Davis W.M. Keck Center.

[2] Wolf, T. (2016). *Magic-Sand: Real-Time Topographic Projection Mapping on Kinetic Sand*. Repositorio de código abierto en GitHub: https://github.com/thomwolf/magic-sand.

[3] Grupo de Investigación Multimedia Interactiva y Animación Digital (2026). *Plan de Trabajo y Cronograma: Gamificación y Seguimiento de Rutas en Entorno AR-Sandbox*. Universidad Distrital Francisco José de Caldas.